

CS 589 HW4

Hyperparameter Search

Introduction

- This assignment is due on **Dec 4th** at 11:59 pm EST.
- **Collaboration or plagiarism will result in a 0 and being reported to the Graduate Academic Integrity Board. This includes all sorts of online resources and AI tools.**
- Please submit the code only after checking it. If the code does not compile, the assignment will be graded as 0.

Problem Description

In the previous homework (HW3), a BERT model was trained to predict tags for Stack Overflow posts. In HW3, you used the default hyperparameter setting for learning rate, batch size, etc. The goal of this assignment is to learn how we can automatically search these hyperparameters so that the model can achieve better performance.

Settings

In this assignment, each trial will follow the same setting as Homework 3 (except for different hyperparameter). That is, we will apply the same task (StackOverflow for tag prediction), use the same model (**bert-base-uncased**), we will use the T4 GPU in Google CoLab.

Hyperparameters to Tune

We will tune the **learning rate** and **number of epochs**. For each hyperparameter, we will train the **bert-base-uncased** model using the training data, and report the prediction NDCG score on the validation data as the criterion to select the best hyperparameter. The **batch size** is fixed to 4 for lowering the memory cost.

The execution of each hyperparameter is called a trial. A trial is defined as a single execution of one hyperparameter for training → getting the validation NDCG for that hyperparameter.

Task 1: Using Grid Search to Search Hyperparameter (33 points)

You will implement the grid search algorithm, which exhaustively searches through a predefined hyperparameter space, in this case, the grid search space we search is (batch size is fixed to 4):

- learning rate: {1e-7, 1e-6, 1e-5}
- epochs: {1,2,3,4,5}

Taks 2: Using FLAML to Search Hyperparameter (33 points)

FLAML (Fast and Lightweight AutoML) is a Python library developed by Microsoft Research that simplifies the process of automating machine learning tasks, including automated hyperparameter search. You will need to read FLAML's documentation to get familiar with how to use it for hyperparameter search. <https://microsoft.github.io/FLAML/>.

More specifically, you will follow the example in 'Tune User Defined Function' (i.e., <https://microsoft.github.io/FLAML/docs/Use-Cases/Tune-User-Defined-Function>) to implement the automatic hyperparameter search for the tag prediction in HW3 (i.e., learning rate and number of epochs).

FLAML's search space (e.g., `config=config_search_space`) is the following. The batch size is fixed to 4.

- learning rate: `tune.loguniform(1e-7, 1e-5)`
- epochs: `tune.randint(1,6)`

Since grid search searched for 15 hyperparameters, in FLAML you will also search for 15 hyperparameters, i.e., set the `num_samples` of `tune.run` to 15.

Set `resources_per_trial={"gpu": 1}`, i.e., each trial uses 1 GPU.

Set `n_concurrent_trials=1`. Because we only have 1 GPU, there only exists 1 trial the entire time. Therefore, the hyperparameter search will be done *sequentially*. That is, we only try the next hyperparameter after the previous trial has finished, there never exist two trials running at the same time.

Set the starting point of FLAML `low_cost_partial_config={"n_epoch": 1}`. FLAML starts with hyperparameters that have low cost and gradually move to hyperparameters of higher costs. By setting this starting point, you force FLAML to search for a hyperparameter with `epoch = 1` in the first trial.

Task 3: Compare Grid Search and FLAML by Plotting the Best NDCG (33 points)

Plot FLAML and Grid Search best NDCG score over wall clock time on the same plot, where the x axis is the wall clock time:

$$WCT = current_time - starting_time$$

and the y axis is the best validation NDCG *so far* using either FLAML or Grid Search (Note: therefore the NDCG must be non-decreasing over time). More specifically, the plot

should look like Figure 1, where the x value of the i-th dot is the ending time of the i-th trial, and the y value of the i-th dot is the best NDCG over trial $1, \dots, i$.

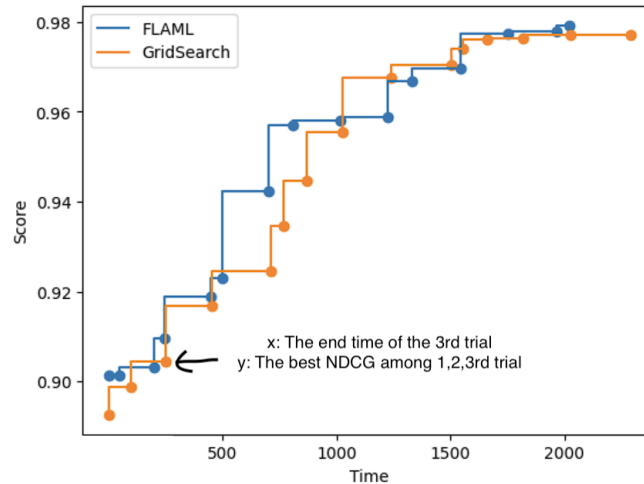


Figure 1: Best NDCG score (y axis) vs Wall Clock Time (x axis)

Hints

Hints for Assignment 4:

1. An example of how to use `tune.run` of FLAML (tune user-defined function) can be found in the page of <https://microsoft.github.io/FLAML/docs/Use-Cases/Tune-User-Defined-Function/> by searching `obj_from_resource_attr` (Figure 2). Essentially, you need to move the execution of train/validation of HW3 to a function, then pass the function along with the search space, etc. to `tune.run`.
2. How to obtain the wall clock time for each trial: record the starting time `t_start`, then use the current time `time.time() - t_start` as the WCT. At the end of the function for each trial, return the WCT along with the validation NDCG, i.e., at the end of Figure 2, return the WCT. Then after `tune.run` finishes and returns `analysis`, you can obtain the WCT and NDCG scores from `analysis`:

```
for each_trial in analysis.trials:
    print(each_trial.last_result)
```

Late policy

90% if late by 1 date, 70 % if late by 2 days, 50 % if late by 5 days and 0 there after.

```
def obj_from_resource_attr(resource_attr, X_train, X_test, y_train, y_test, config):
    from lightgbm import LGBMClassifier
    from sklearn.metrics import accuracy_score

    # in this example sample size is our resource dimension
    resource = int(config[resource_attr])
    sampled_X_train = X_train.iloc[:resource]
    sampled_y_train = y_train[:resource]

    # construct a LGBM model from the config
    # note that you need to first remove the resource_attr field
    # from the config as it is not part of the original search space
    model_config = config.copy()
    del model_config[resource_attr]
    model = LGBMClassifier(**model_config)

    model.fit(sampled_X_train, sampled_y_train)
    y_test_predict = model.predict(X_test)
    test_loss = 1.0 - accuracy_score(y_test, y_test_predict)
    return {resource_attr: resource, "loss": test_loss}
```

Figure 2: An example of the function for `tune.run`

Submission Details:

- Submit the notebook file, make sure the plot is visible.
- Submit a txt file explaining your finding for comparing Grid Search and FLAML.